

Multiway models

Mathies Brinks Sørensen

DTU

02582 Computational Data Analysis, 2025

Today's lecture

- Introduction
- Tensors mathematics
- Tucker model and examples
- PARAFAC model and examples
- PARAFAC VS Tucker
- Model selection

Introduction

Multiway data

To build multiway models, we need multiway data structures

Common data structures consist of rows and columns (2D)

A multiway data structure consists of **a minimum** of multiple 2D matrices that can be made into multiway models

Multiway data

vector
order-1 tensor

x



x_i

matrix
order-2 tensor

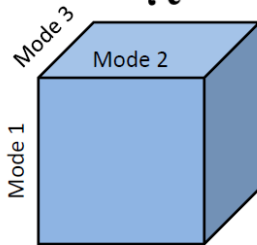
X



$x_{i,j}$

3-way array
order-3 tensor

$\mathcal{X}$



$x_{i,j,k}$

Tensors - The building blocks of Multiway models

Multiway models consist of "building blocks" called tensors

Tensor fancy mathematics definition:

- An algebraic object that describes a **multilinear** relationship between sets of algebraic objects related to a vector space

A simple definition:

- A tensor is an N-dimensional array used to represent data, where N is the number of axes or modes

Tensors - The building blocks of Multiway models

Tensors sound dangerous, but you may have encountered them already!



Tensors - The building blocks of Multiway models

Examples of domains where multiway data structure is applicable

- Psychology
- Chemometrics
- Bioinformatics
- Computer vision

However, the multiway data structure is often ignored in favor of traditional 2D data analysis!

Tensors - The building blocks of Multiway models

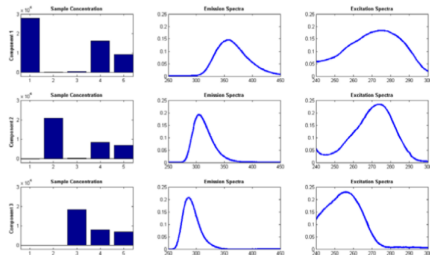
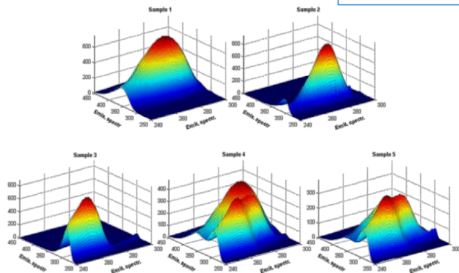
Why do we even look at tensors?

- Uniqueness of the decomposition without additional constraints
- Identify components even when facing very poor signal to noise ratios (SNR)
- Handle data sets with large amounts of missing data
- Explicitly take into account the multi-way structure of the data
- Increased interpretability

Multiway models application in Chemometrics

"Second order advantage, making it possible to do quantitative chemical analytes even in the presence of un-calibrated interferents"

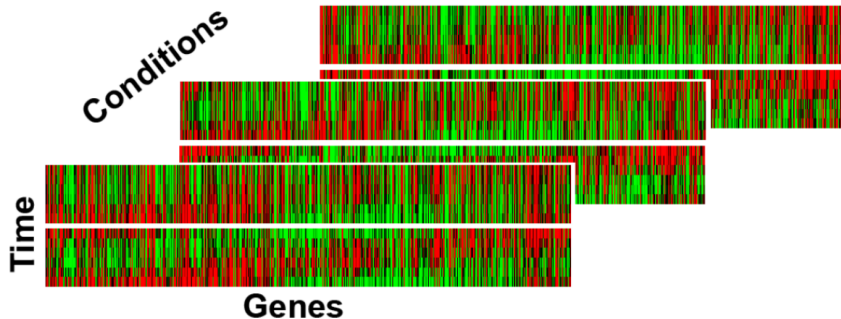
Beer-Lambert's law stating a linear relation between absorbance of light and the concentration of a compound



$$\chi^{Excitation \times Emission \times Samples} \approx \sum_{d=1}^{D_{components}} a_d^{Excitation} \circ b_d^{Emission} \circ c_d^{Samples}$$

Multiway models application in Bioinformatics

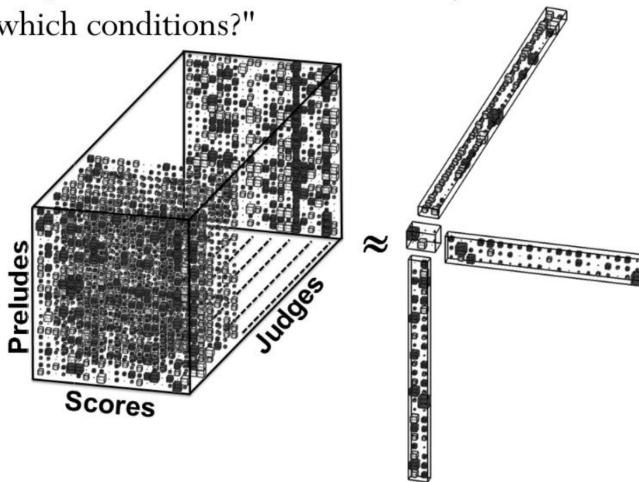
In micro-array analysis tensor decomposition useful in order to remove experimental artifacts and detect distinct stimulus dependent sets of functionally related genes



$$\chi^{Gene \times Time \times Condition} \approx \sum_{lmn} g_{lmn} \mathbf{a}_l^{Gene} \circ \mathbf{b}_m^{Time} \circ \mathbf{c}_n^{Condition}$$

Multiway models application in Psychology

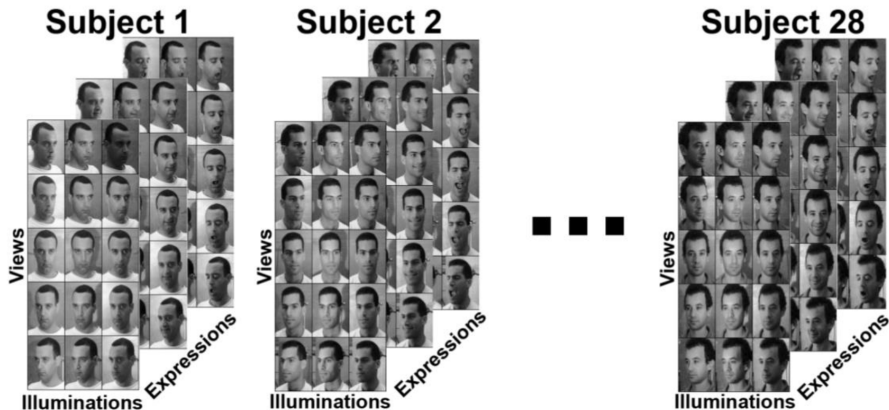
"which group of subjects behave differently on which variables under which conditions?"



$$\chi^{Prelude \times Score \times Subject} \approx \sum_{lmn} g_{lmn} \mathbf{a}_l^{Prelude} \circ \mathbf{b}_m^{Score} \circ \mathbf{c}_n^{Subject}$$

Multiway models application in Computer vision

Extraction of patterns that generalize well across modes of variation "TensorFaces" demonstrated to be significantly more accurate than PCA as multiple interrelated subspaces can collaborate to discriminate different classes



$$\chi^{People \times Views \times Illum. \times Expres. \times Pixels} \approx \mathcal{G} \times_1 \mathbf{A}^{People} \times_2 \mathbf{B}^{Views} \times_3 \mathbf{C}^{Illum.} \times_4 \mathbf{D}^{Expres.} \times \mathbf{E}^{Pixels}$$

Software for multiway models



- **Tensorly** with backends for numpy, pytorch, tensorflow and mxnet
- **Scikit-tensor** (Python2)



MATLAB

- **N-WAY toolbox** (free)
- **PLS toolbox**
- **Tensor toolbox** (free)



- **multiway** package
- **rTensor** package

Tensor Mathematics

Tensor notation

- Tensors are denoted using calligraphic letters

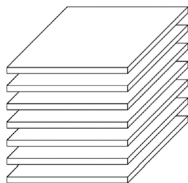
$$\mathcal{X} \in \mathbb{R}^{\mathcal{I}_1 \times \mathcal{I}_2 \times \dots \times \mathcal{I}_N} \quad (1)$$

$$\mathcal{X}^{\mathcal{I}_1 \times \mathcal{I}_2 \times \dots \times \mathcal{I}_N} \quad (2)$$

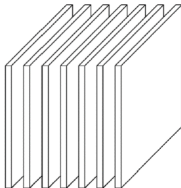
- N denotes the order of the tensor
- $\mathcal{X}$ is the measured response across the tensor
- $\mathcal{I}_N$ denotes the dimensionality of the n -th mode (with size of each mode being equal to the number of elements)

3 way Tensors

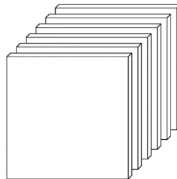
How to view the dimensions of a 3-way tensor:



Horizontal $\mathcal{A}(i, :, :)$

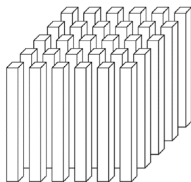


Lateral $\mathcal{A}(:, j, :)$

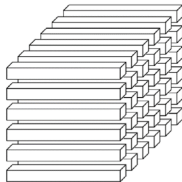


Frontal $\mathcal{A}(:, :, k)$

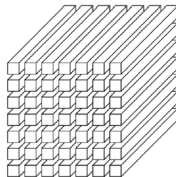
slices or slabs



columns $\mathcal{A}(:, j, k)$



rows $\mathcal{A}(i, :, k)$



tubes $\mathcal{A}(i, j, :)$

fibers

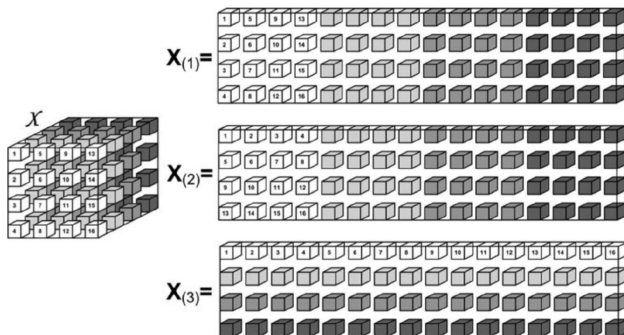
3 way Tensors

- The Frobenius norm of a tensor $\mathcal{A}$ is defined such that:

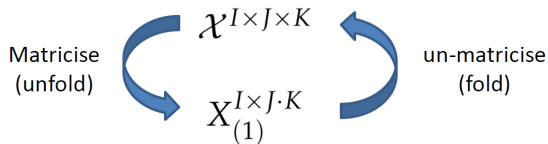
$$\|\mathcal{A}\|_F = \sqrt{\sum_i \sum_j \sum_k a_{ijk}^2} \quad (3)$$

3 way Tensors

Folding and unfolding tensors



Mode1 example



3 way Tensors

In Python, this is the equivalent of reshape in numpy.

N-mode multiplication

The n -mode multiplication of an order N tensor $\mathcal{X}^{I_1 \times I_2 \times \dots \times I_N}$ with a matrix $M^{J \times I_n}$ step by step:

Step 1: Unfold the tensor $\mathcal{X}$ along n (we have a matrix)

$$\mathcal{X}_n \in \mathbb{R}^{I_n \times (I_1 \times \dots \times I_{n-1} \times I_{n+1} \times \dots \times I_N)} \quad (4)$$

Step 2: Multiply along the n -mode (still a matrix)

$$\mathcal{X} \times_n M = \mathcal{Z}^{J \times (I_1 \times \dots \times I_{n-1} \times I_{n+1} \times \dots \times I_N)} \quad (5)$$

Step 3: fold back to a tensor

$$\mathcal{X} \times_n M = \mathcal{Z}^{I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N} \quad (6)$$

The complete operation is defined as:

$$[\mathcal{X} \times_n M]_{(n)} = MX_{(n)} \quad (7)$$

N-mode multiplication

The 3 steps are automated in Python to make life easier with the functions `mod_dot()` and `multi_mode_dot()`

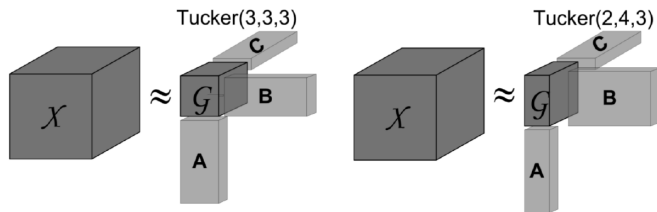
`mod_dot()` is equal to eq 7 (unfold tensor, multiply, fold back)

`multi_mode_dot()` is the `mod_dot()` in sequence: (unfold, multiply, fold back, unfold, multiply, fold back, etc.)

Tucker model and examples

Tucker 3

- proposed by Tucker in 1970
- in these slides: TUCKER3 as decomposition of a 3rd order tensor $\mathcal{X}^{I \times J \times K}$



- $\mathcal{G}$ is the core tensor
- The dimensionality of $\mathcal{G}$ defines the ranks in the respective modes 1, 2, and 3 (P, Q, and R).

Tucker 3

Outer product representation:

$$\mathcal{X} \approx \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} a_p \circ b_q \circ c_r = \mathcal{G} \times_1 A \times_2 B \times_3 C$$

As sum of outer
vector products
scaled by core
elements g_{pqr}

n-mode
multiplication using
loading matrices A,
B and C

This is estimated using eq 7 in sequence along modes 1, 2, and 3

Tucker 3

Scalar form representation:

$$x_{ijk} \approx \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} a_{ip} b_{jq} c_{kr}$$

Same results as the outer product, but (maybe) easier to see what's going on If you don't know tensor notation

Tucker 3

Matrix form representation:

$$X_{(1)} \approx AG_{(1)}(C \otimes B)^T$$

$$X_{(2)} \approx BG_{(2)}(C \otimes A)^T$$

$$X_{(3)} \approx CG_{(3)}(B \otimes A)^T$$

The **Kronecker** product: Its operation yields all possible outer vector products between columns in C and B.

Excellent for optimization purposes, as if we define Z as:

$$Z_A = G_{(1)}(C \otimes B)^T \quad (8)$$

We can estimate A from the least squares solution:

$$A = X_{(1)}Z_A^T(Z_AZ_A^T)^{-1} \quad (9)$$

Tucker 3

What is the goal of all this?

- Making a meaningful decomposition of complex multi-way data in a low-dimensional space

How can we make a meaningful decomposition through optimization?

Tucker 3 optimization

The loss function :

$$\min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\|_F^2, \quad \text{where} \quad \hat{\mathcal{X}} \approx \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} a_p \circ b_q \circ c_r$$

Minimize the loss function via alternating least squares (ALS)

$$A \leftarrow X_{(1)} \left(G_{(1)} (C \otimes B)^T \right)^\dagger = X_{(1)} Z_A^\dagger = X_{(1)} Z_A^T (Z_A Z_A^T)^{-1} \quad (10)$$

$$B \leftarrow X_{(2)} \left(G_{(2)} (C \otimes A)^T \right)^\dagger = X_{(2)} Z_B^\dagger = X_{(2)} Z_B^T (Z_B Z_B^T)^{-1} \quad (11)$$

$$C \leftarrow X_{(3)} \left(G_{(3)} (B \otimes A)^T \right)^\dagger = X_{(3)} Z_C^\dagger = X_{(3)} Z_C^T (Z_C Z_C^T)^{-1} \quad (12)$$

Upon convergence: $\mathcal{G} \leftarrow \mathcal{X} \times_1 A^{-1} \times_2 B^{-1} \times_3 C^{-1}$

Initialize parameters: Random guesses or higher-order SVD.

Other tricks: Imposing non-negativity

Note that the $\dagger$ is the Moore Penrose inverse to handle singularities

Tucker 3 example

TUCKER for image reconstruction (compression task)

Tucker 3 example

The data



Tucker 3 example

The results

original



Tucker 3



Residuals

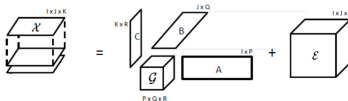
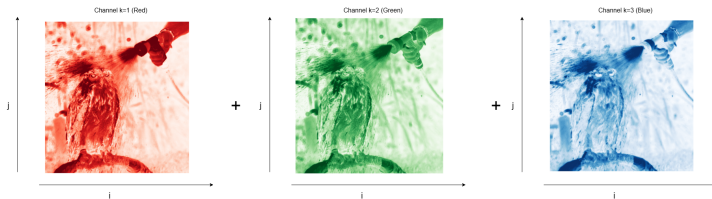


Tucker 3 example

The data structure

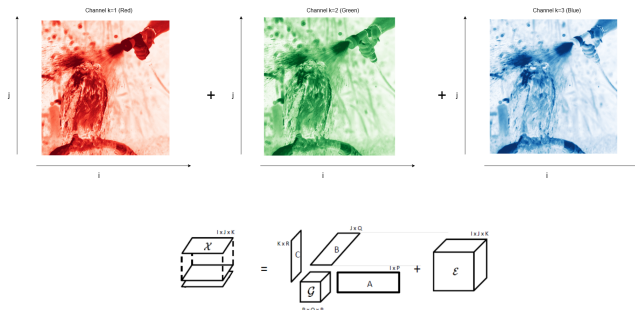


=



Tucker 3 example

The data structure



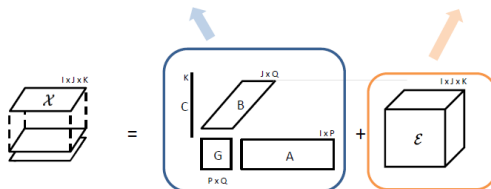
- Dimensionality of $\mathcal{G}$ defines ranks in the modes
- $\mathcal{G}$ defines all possible cross-talk options between A, B, and C loadings

Tucker 3 example

The Tucker reconstruction: The effect of increasing P , Q , and R , let's see!

Tip: Ideally, we want to have a low ranking mode - in this case, channels, as we cannot meaningfully go beyond 3!

Tucker 3 example



P = 200
Q = 200
R = 1

We managed to isolate colors!

Tucker 3 example

Can we interpret the core tensor $\mathcal{G}$ at R=1

	0	1	2	3	4	5	6	7	8	9	10
0	71699.8	-0.000153222	-4.6534e-05	1.47754e-05	0.000239793	0.000162192	1.29321e-05	-5.17378e-05	-0.000148658	-6.89982e-05	3.65918e-05
1	-0.00029968	20982.6	2.0431e-05	-1.78987e-05	0.000214309	-5.43825e-05	-8.09544e-05	-4.06325e-05	5.01248e-05	7.87505e-05	2.0929e-05
2	-0.000451843	0.000304858	16087.6	-3.69055e-06	-0.000252339	-0.000182868	-2.8621e-05	6.40718e-05	8.06302e-05	9.2384e-05	1.04174e-05
3	0.000204362	-5.54977e-05	9.39874e-05	13283.4	3.89458e-05	-2.18968e-05	-4.03036e-05	-6.23735e-06	-5.74139e-05	-6.22782e-05	-6.78912e-05
4	-0.000534137	9.68665e-06	0.000139812	-5.24717e-05	-10639.1	0.000112541	-7.07728e-05	-7.88216e-06	-4.83326e-05	-4.85624e-05	-0.000109905
5	0.000470753	-0.000160167	-5.19824e-05	3.10396e-05	-1.15473e-05	9330.96	-1.16499e-05	-8.92184e-05	-5.41198e-05	-2.35649e-05	-3.19711e-05
6	0.000292131	-0.000309672	-1.33087e-05	3.46693e-05	-7.72359e-06	-0.000142035	8315.82	-6.1613e-05	3.36074e-05	6.35971e-05	-1.78511e-05
7	0.000153227	-0.00027453	-2.41777e-05	-9.10836e-05	5.71206e-05	3.42201e-05	3.85908e-05	7162.57	5.02224e-05	0.000124248	6.17429e-05
8	-8.82435e-05	0.000233565	8.03997e-06	0.000129066	-4.08118e-05	-1.67875e-05	1.04743e-05	-0.000116847	5710.66	-4.2591e-05	1.77779e-05
9	-0.000181394	0.000107958	-2.97427e-05	8.75909e-06	-4.98091e-05	4.39669e-05	-1.55151e-05	-3.58236e-05	-9.63205e-05	-5562.91	-1.66182e-05
10	0.000113468	-9.74171e-05	-3.06336e-05	-1.48004e-05	-3.52096e-05	-2.23154e-07	3.45541e-05	9.34562e-05	5.16051e-05	3.39962e-05	-5150.52
11	-6.05648e-05	7.95885e-05	-4.42424e-06	8.30994e-05	0.000115835	-6.84602e-06	-5.24916e-05	-1.52195e-05	-7.0824e-05	1.22106e-05	2.55412e-05
12	5.66992e-05	-8.83137e-05	-3.80139e-05	-4.78281e-05	4.43704e-05	-4.9746e-05	-2.40062e-05	6.18954e-05	1.88337e-05	1.14623e-05	2.49675e-05
13	0.000193922	-1.8025e-05	-2.85895e-05	-3.19965e-05	-4.1027e-05	3.02868e-05	3.99925e-05	3.03184e-05	9.22804e-05	3.19641e-05	4.48956e-05
14	1.46037e-05	5.32655e-05	-2.01291e-05	3.21087e-05	-1.69938e-05	7.91575e-07	-2.88915e-05	5.01176e-05	-1.7656e-05	5.92543e-05	2.91951e-05
15	0.00021286	-6.15217e-05	-2.9029e-05	6.24044e-06	-1.89661e-05	2.04256e-05	-2.36006e-05	2.22069e-05	7.14476e-05	6.19694e-05	4.51171e-05
16	-9.05755e-05	1.46796e-05	-8.38914e-06	-7.43311e-05	-5.69886e-06	2.77084e-05	3.00069e-05	-3.00979e-05	2.08291e-05	2.54256e-05	1.59089e-05

Tucker 3 example

What happens at higher ranks of R?

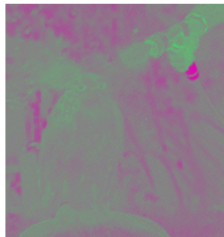
original



Tucker 3



Residuals



original



Tucker 3



Residuals



Tucker 3 example

Can we interpret the core tensor $\mathcal{G}$ at R=3

	0	1	2	3	4	5	6	7
0	71699.7	71.1224	59.1349	-37.0495	11.4963	25.1825	-18.044	-36.2471
1	55.1005	20951	-602.931	-365.659	223.901	1.90502	-132.59	228.35
2	-17.6775	593.796	16056.6	-366.773	-101.110	-237.086	-33.9244	-85.0199
3	-1.70077	457.416	410.638	12247.5	428.754	243.005	187.064	47.2405
4	-31.5783	-188.486	-295.933	613.983	-10379.1	-1009.96	-1137.56	-285.181
5	20.4551	-21.0963	413.680	-276.688	-1773.57	8568.6	1692.71	680.166
6	-2.93400	6.72628	83.7482	-121.116	-1019.96	-2107.74	7850.22	927.735
7	19.4393	-249.457	261.511	116.91	-160.049	-493.236	-1437.72	6740.4
8	-2.08522	159.215	-166.404	-135.635	94.7963	175.113	782.42	1660.24
9	-10.2369	4.57252	97.1063	91.5036	259.775	236.525	317.961	-450.913
10	25.1151	27.9883	39.0125	46.1743	-165.184	-35.5421	-261.968	-652.272
11	24.8001	89.8433	13.9043	117.082	-260.819	-214.588	-90.3435	-112.809
12	17.0002	5.54534	15.5251	5.08553	-157.931	-147.51	-152.433	-424.604
13	8.74056	178.346	-84.2706	-59.778	-329.571	-80.2703	-304.388	-176.148
14	-10.9143	-115.648	180.185	112.987	205.406	150.801	232.164	42.0801
15	11.3554	26.2728	101.227	-51.0025	-230.004	-243.136	-203.031	-103.813
16	-3.88004	43.1506	-53.5029	-104.049	-50.0042	14.0443	-130.561	160.514
17	1.01214	-65.2014	173.139	-67.4124	-44.9279	-160.794	-50.6338	-212.965
18	3.24012	45.8345	52.0705	-54.3649	-164.725	-167.21	-177.642	-80.7122

	0	1	2	3	4	5	6	7
0	-37.0958	648.749	215.322	0.766163	1037.3	923.217	-337.282	-340.236
1	-620.702	-2141.05	-79.9781	-250.514	1300.06	-56.1775	-757.61	110.11
2	-936.072	1608.17	-5.40536	158.035	-2210.43	-930.400	87.739	178.582
3	873.519	148.274	824.911	1737.86	-59.6189	-174.664	-193.15	-2.35702
4	-3010.32	-46.1832	599.305	-23.259	-62.3054	786.4	-499.737	-241.975
5	1057.38	-1236.05	-111.192	-130.342	-154.86	-11.7786	-244.3	-180.87
6	1143.75	-1235.98	40.7121	5.36202	-161.434	-594.11	105.747	447.118
7	677.46	-1410.1	-56.4776	-340.193	329.991	-25.0225	-1.35229	993.794
8	-603.117	1035.92	91.2071	521.302	-151.623	109.361	95.1338	-590.365
9	-470.804	-135.839	-2.52014	-207.240	-42.473	200.803	-120.395	-174.802
10	600.430	-350.55	-240.35	66.6704	-210.735	-51.1908	75.427	471.442
11	470.247	-326.313	-206.095	-344.163	-548.566	-51.6004	258.932	-17.8302
12	526.765	-303.580	-259.223	180.406	172.762	-185.857	-270.953	430.773
13	1164.57	461.704	-327.035	252.624	-299.14	115.544	189.84	368.115
14	-645.572	-353.948	72.2003	-326.264	293.521	142.827	419.209	-560.095
15	676.739	-250.871	-69.6595	-222.667	140.934	-24.0923	67.7358	-143.84
16	343.953	311.728	-50.3082	428.002	14.0394	-159.331	-145.767	333.403
17	210.757	-505.205	212.975	-11.0948	-151.456	-8.58254	100.574	-62.2611
18	599.946	-100.867	-137.181	72.787	-251.5	-200.409	150.800	66.5343

	0	1	2	3	4	5	6	7
0	1553.5	1927.56	572.686	-102.003	308.556	117.002	-544.786	23.467
1	570.393	-2971.96	-229.356	-88.1076	255.78	292.265	-415.169	353.722
2	2020.71	-123.518	-909.622	416.251	187.923	490	437.97	-556.782
3	-385.708	500.831	463.725	-1414.17	320.792	-23.2805	-0.408291	114.483
4	447.743	-655.28	-271.384	351.449	621.78	41.4701	-134.072	-247.881
5	-585.835	-8.2139	9.07089	-205.917	170.701	-0.4500279	-222.534	382.01
6	90.1273	693.448	41.4588	-0.371103	-44.4084	140.636	-140.955	371.377
7	8.53162	132.437	40.2298	308.47	107.845	-200.622	-192.388	-610.714
8	-199.3	-121.657	167.976	-69.3026	-26.1446	179.721	43.68	78.6618
9	-121.39	-104.135	100.016	-90.1523	1.49711	-1.75105	-75.64	-42.1932
10	24.0429	343.767	2.2965	170.909	243.870	-67.8674	-45.4926	67.0314
11	-13.9347	60.6911	-221.726	100.234	149.055	-92.3717	31.6291	-107.18
12	-47.3005	-112.259	96.9554	522.259	9.30033	131.45	-42.7051	11.0401
13	135.666	283.514	-45.481	254.037	39.4318	-55.0804	143.505	44.2966
14	-24.794	-64.2602	-509.478	87.5111	216.432	-45.2261	-56.1271	30.6437
15	-127.573	155.153	-39.6346	-217.62	29.1070	-33.9508	85.0953	55.0273

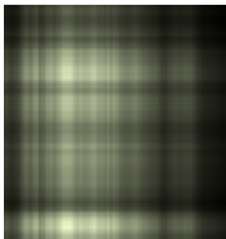
Tucker 3 example

Let us see what happens when Q and P increase from 1 to 200. $P=1$, $Q=1$, $R=3$

original



Tucker 3



Residuals



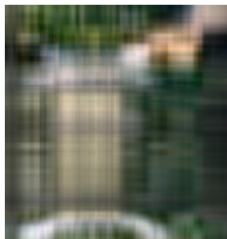
Tucker 3 example

Let us see what happens when Q and P increase from 1 to 200. $P=5$,
 $Q=5$, $R=3$

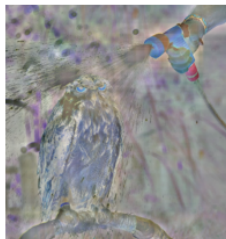
original



Tucker 3



Residuals



Tucker 3 example

Let us see what happens when Q and P increase from 1 to 200. $P=20$, $Q=20$, $R=3$

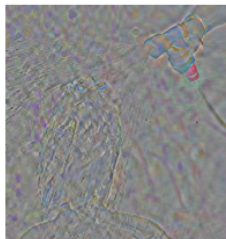
original



Tucker 3



Residuals



Tucker 3 example

Let us see what happens when Q and P increase from 1 to 200.
 $P=100$, $Q=100$, $R=3$

original



Tucker 3



Residuals



Tucker 3 example

Let us see what happens when Q and P increase from 1 to 200.
P=200, Q=200, R=3

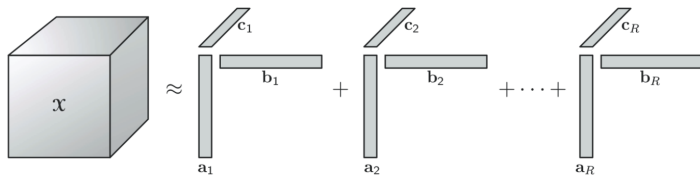


- it turns out what you are seeing is the product between Tensors $B_p C_q^T$ scaled with the core tensor $\mathcal{G}$

PARAFAC model and examples

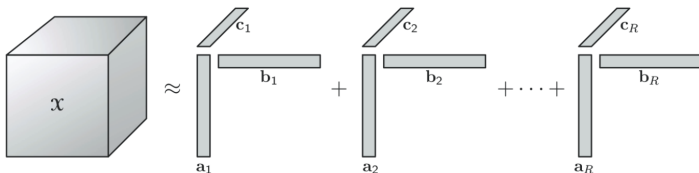
PARAFAC

- Parallel Factor Analysis (PARAFAC)
- Independently proposed by Harshman and Carroll & Chang 1970
- Also called Canonical Decomposition (CP)



- PARAFAC decomposes a tensor into a sum of component rank-one tensors
- R is the overall tensor rank (number of components)

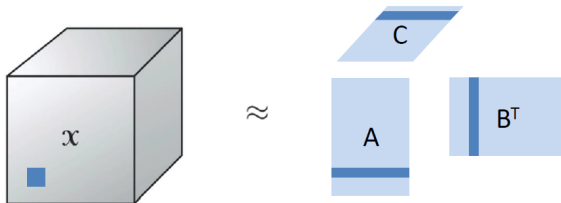
Outer product representation:



$$\mathcal{X} \approx \sum_{r=1}^R a_r \circ b_r \circ c_r$$

PARAFAC

Scalar form representation:



$$x_{ijk} \approx \sum_{r=1}^R a_{ir} b_{jr} c_{kr}$$

PARAFAC

Matrix form representation:

$$X^{I \times J \times K} \approx A (C \odot B)^T$$

$$X_{(1)} \approx A (C \odot B)^T$$

Matricised tensor Khatri-Rao product

Same trick as before, we need the least squares

$$Z_A = (C \odot B)^T \quad (13)$$

$$A = X_{(1)} Z_A^T (Z_A Z_A^T)^{-1} \quad (14)$$

PARAFAC optimization

The loss function :

$$\min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\|_F^2, \quad \text{where} \quad \hat{\mathcal{X}} = \sum_{r=1}^R a_r \circ b_r \circ c_r$$

Minimize the loss function via alternating least squares (ALS)

$$A \leftarrow X_{(1)} (C \odot B) \left(C^T C * B^T B \right)^{-1} = X_{(1)} Z_A (Z_A Z_A^T)^{-1} \quad (15)$$

$$B \leftarrow X_{(2)} (C \odot A) \left(C^T C * A^T A \right)^{-1} = X_{(2)} Z_B (Z_B Z_B^T)^{-1} \quad (16)$$

$$C \leftarrow X_{(3)} (C \odot A) \left(B^T B * A^T A \right)^{-1} = X_{(3)} Z_C (Z_C Z_C^T)^{-1} \quad (17)$$

Note that $(C^T C * A^T A) = (C \odot A)^T (C \odot A)$

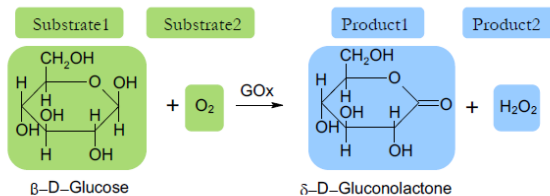
Initialize parameters: Random guess or HOSVD

Other tricks: Impose non-negativity

PARAFAC example 1

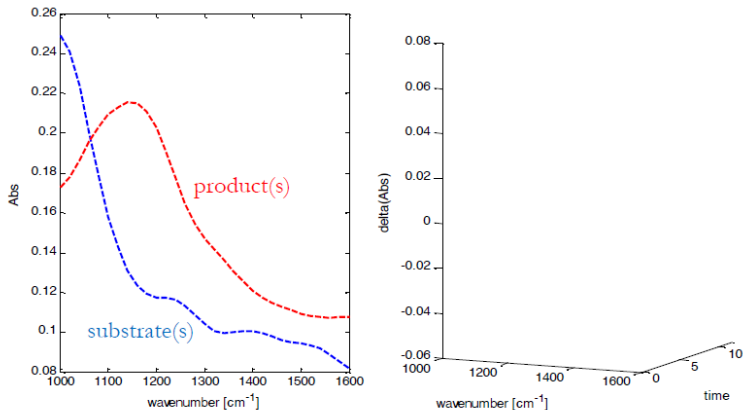
Quantification of enzyme activity rates

PARAFAC example 1



GOx is an Oxido-Reductase

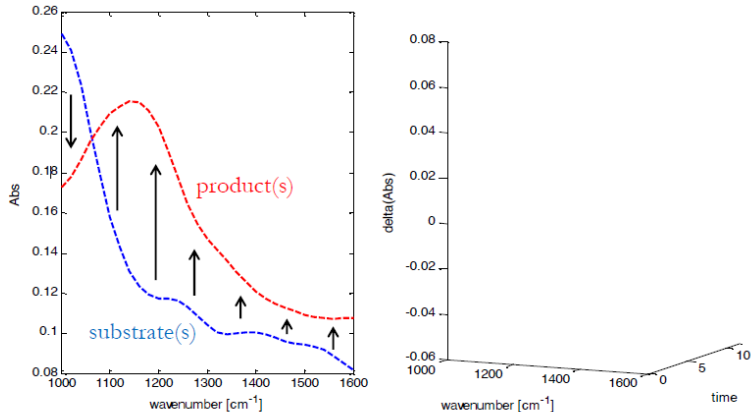
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

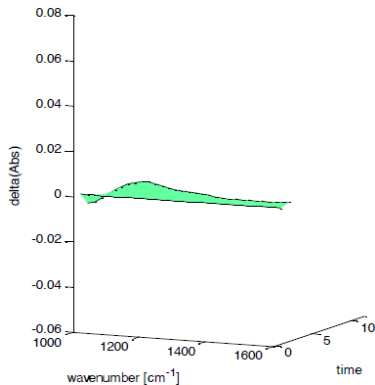
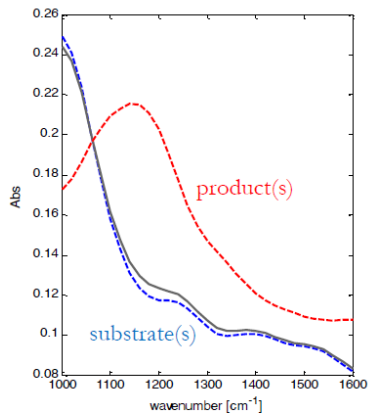
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

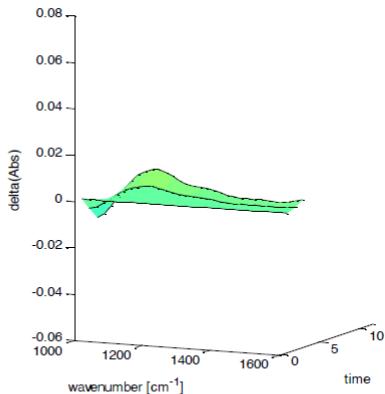
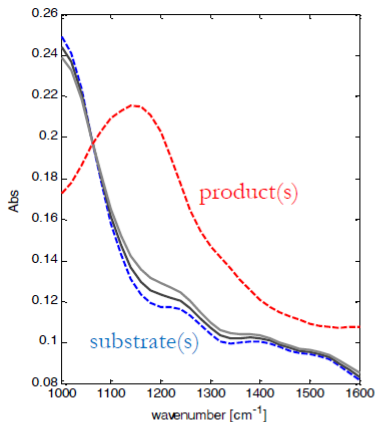
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

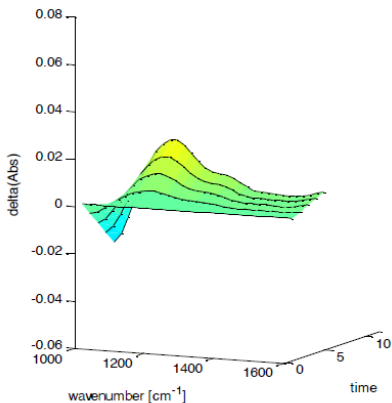
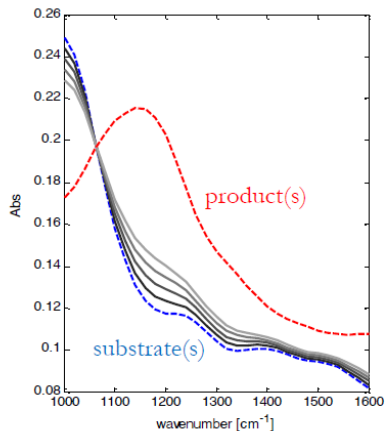
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

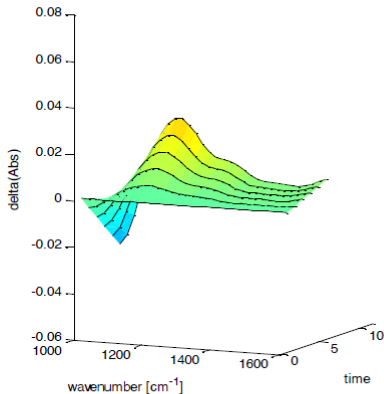
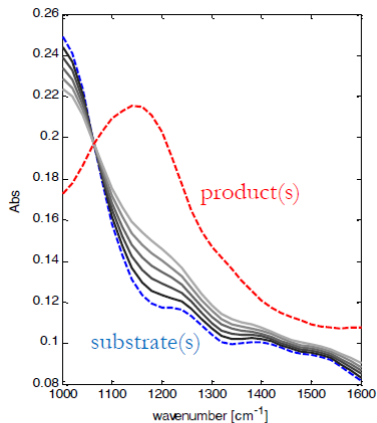
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

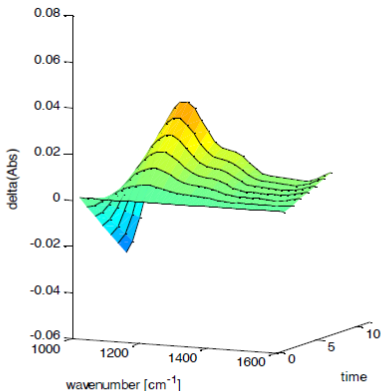
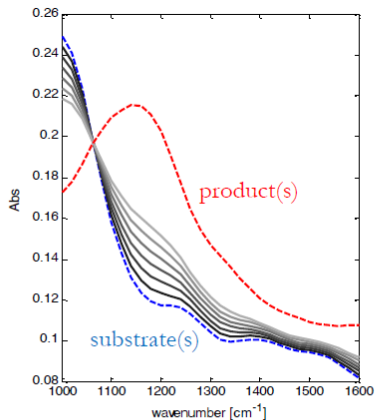
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

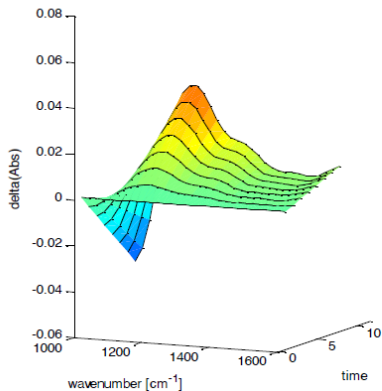
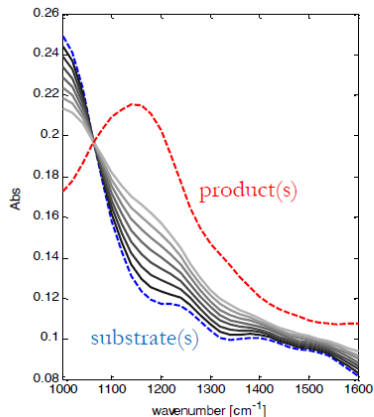
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

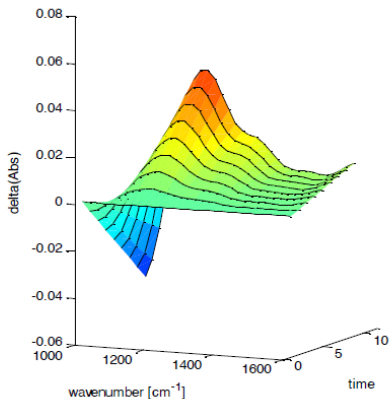
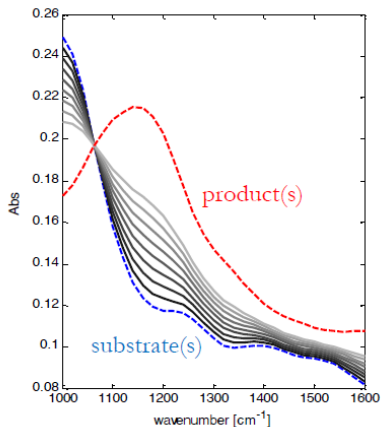
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

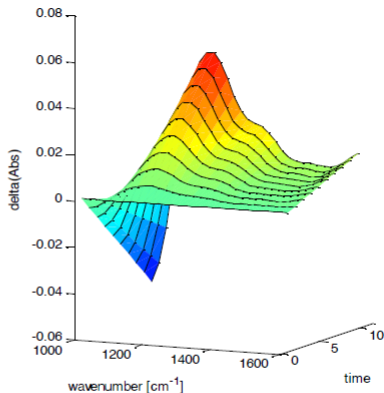
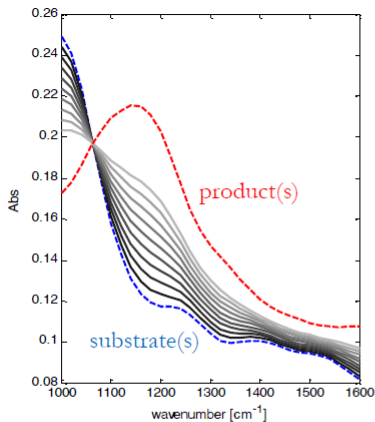
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

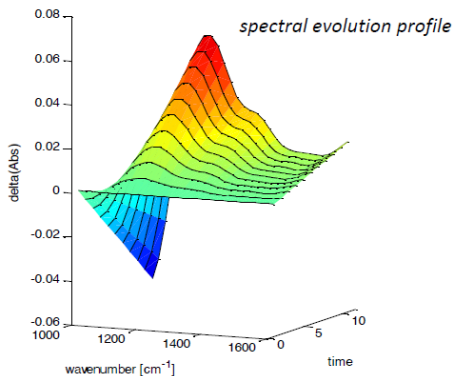
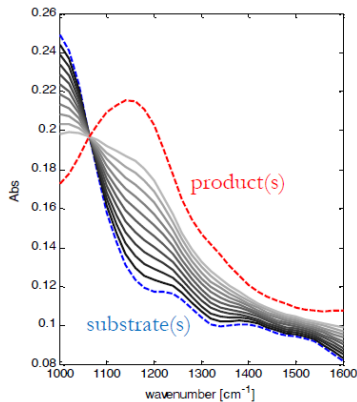
PARAFAC example 1



MAJOR ASSUMPTION:

substrate(s) and product(s) have distinguishable spectral fingerprints

PARAFAC example 1

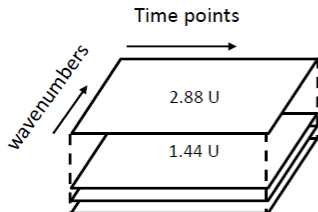


MAJOR ASSUMPTION:

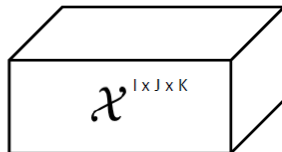
substrate(s) and product(s) have distinguishable spectral fingerprints

PARAFAC example 1 data structure

Two-way data

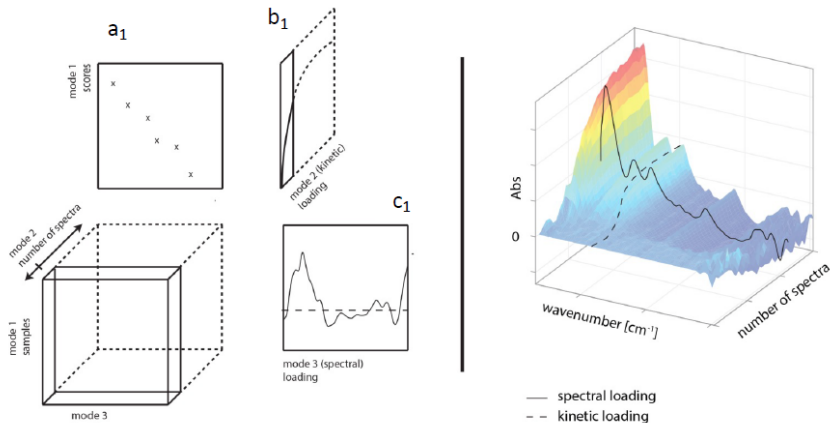


Three-way data



$$\mathcal{X}^{I \times J \times K} = \mathcal{X}^{\text{samples} \times \text{time-points} \times \text{wavenumbers}}$$

PARAFAC example 1 data decomposed

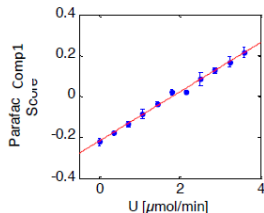


An illustrative decomposition using a single component

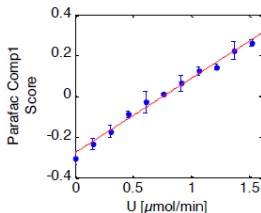
PARAFAC example 1 many models

One PARAFAC model per enzyme

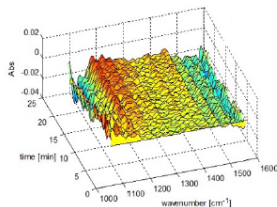
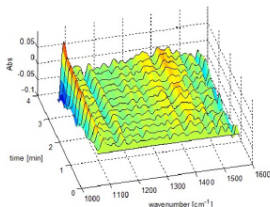
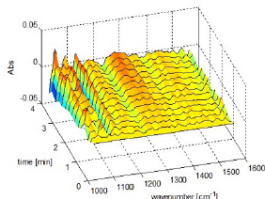
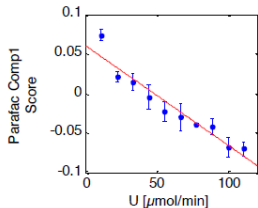
Pectin Lyase



Celluclast 1.5L



Glucose Oxidase

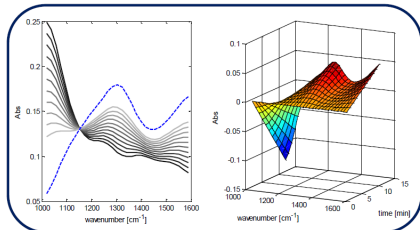


PARAFAC example 2

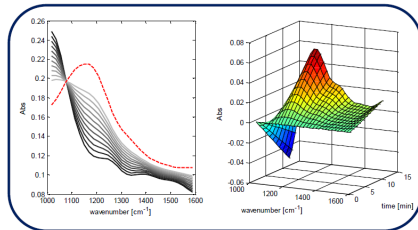
PARAFAC and TUCKER3 for simultaneous quantification of two enzyme activities

PARAFAC example 2

Enzyme A

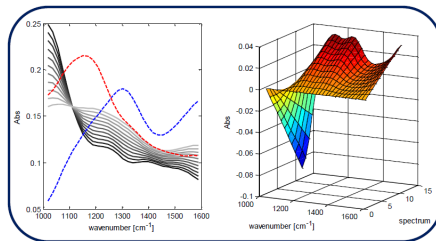


Enzyme B



superimposition

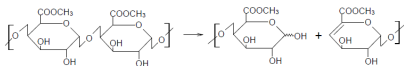
Enzyme A+B



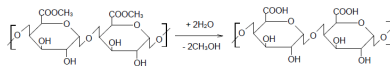
Baum et al. 2013

PARAFAC example 2

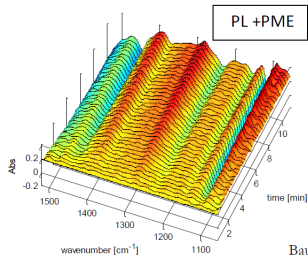
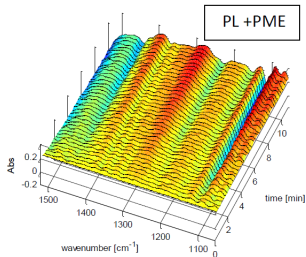
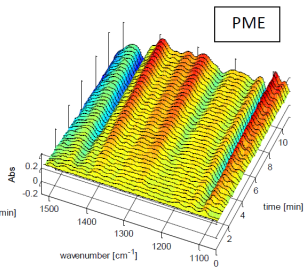
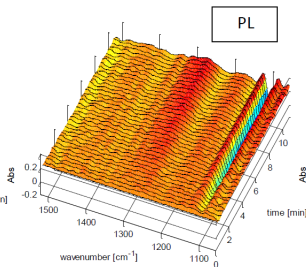
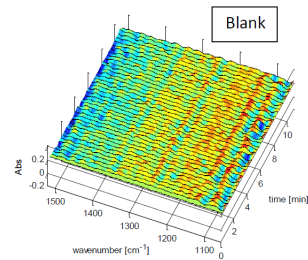
Enzymatic reaction of pectin lyase (PL)



Enzymatic reaction of pectin methyl esterase (PME)



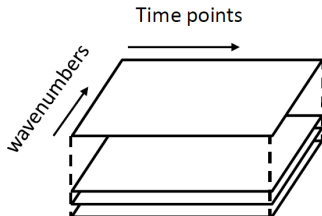
PARAFAC example 2



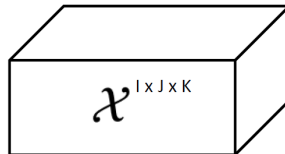
Baum et al. 2013

PARAFAC example 2

Two-way data

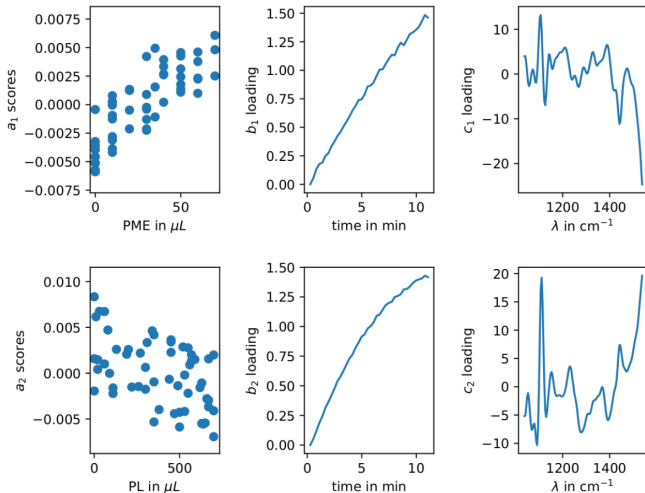


Three-way data



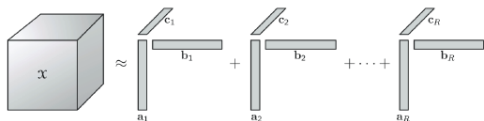
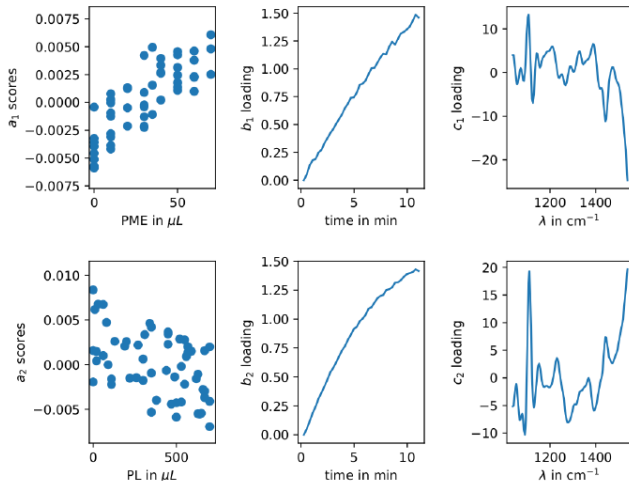
$$\mathcal{X}^{I \times J \times K} = \mathcal{X}^{56\text{samples} \times 40\text{ time-points} \times 131\text{ wavenumbers}}$$

PARAFAC example 2



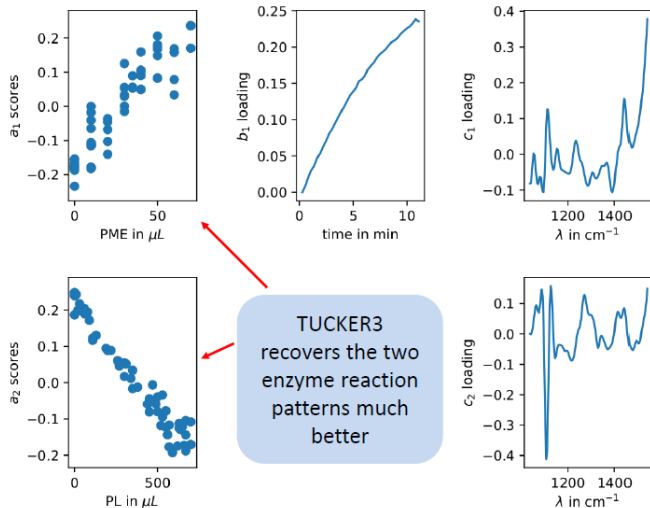
$$\mathcal{X} \approx \sum_{r=1}^R a_r \circ b_r \circ c_r \quad \text{with } R = 2$$

PARAFAC example 2



with $R = 2$

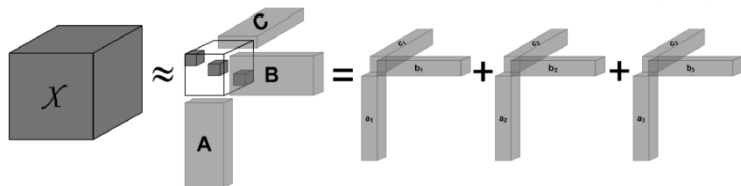
PARAFAC example 2



$$\mathcal{X} \approx \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} a_p \circ b_q \circ c_r \quad \text{with } P = 2, Q = 1 \text{ and } R = 2$$

PARAFAC vs Tucker

PARAFAC vs Tucker



- PARAFAC is a special case of the TUCKER model
- For PARAFAC, the core tensor $\mathcal{G}$, is super-diagonal, meaning it contains ones along its diagonal and zeros otherwise
- TUCKER is good for data compression tasks
- PARAFAC is good for situations where additive (physical) profiles are to be deconvoluted/resolved from each other

PARAFAC vs Tucker

Uniqueness

$$\mathcal{X} \approx (\mathcal{G} \times_1 Q) \times_1 (AQ^{-1}) \times_2 B \times_3 C = \tilde{\mathcal{G}} \times_1 \tilde{A} \times_2 B \times_3 C$$

You can adjust the core of Tucker 3 by any rotation matrix Q and still get the same tensor representation.

Parafac has a diagonal core tensor $\rightarrow$ rotation matrix Q gives a new solution $\rightarrow$ parafac solution is unique

PARAFAC vs Tucker

original



Tucker 3



PARAFAC



Model selection

Model selection

- Core Consistency Diagnostics (CORCONDIA) represents a heuristic for that purpose
- To obtain a valid PARAFAC model, the right number of components has to be found (components are generally not nested)
- For a PARAFAC model with R components we obtain

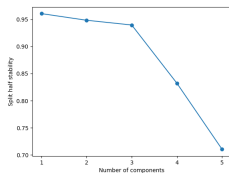
$$CORCONDIA = 100 \cdot \left(1 - \frac{\|\mathcal{I} - \mathcal{G}\|_F^2}{\|\mathcal{I}\|_F^2}\right) \quad (18)$$

- $\mathcal{I}^{R \times R \times R}$ is a perfect super diagonal core tensor

Model selection

Split-half analysis

- Randomly split data into two halves along one mode (usually in the sample mode).
- Evaluate split-half stability via the factor match score (FMS) estimated for the split data at different numbers of components



$$\text{FMS} = \sum_{r=1}^R \frac{a_r^T \hat{a}_r}{\|a_r^T\| \|\hat{a}_r\|} \frac{b_r^T \hat{b}_r}{\|b_r^T\| \|\hat{b}_r\|} \frac{c_r^T \hat{c}_r}{\|c_r^T\| \|\hat{c}_r\|}$$

Compare with CORCONDIA and see if they match!

Model selection

Residual analysis and domain knowledge

Think for yourself

What happens if we do not have a 3D multiway structure? Can we have a parafac or Tucker model with just A and B?

Today's Exercises

- Analyze casein data using a single PARAFAC component.
- Analyze Amino acid fluorescence data and implement CORCONDIA to evaluate suitable PARAFAC models

Questions?

Any questions for today's topic?